

Bring Your Own Assurance: Composable Test and Evaluation for Agentic Computer-Vision Development with CheckMAITE

Anonymous submission

Abstract

AI systems increasingly assist with preparing data, running experiments, and creating candidate models. As these development workflows become more autonomous, candidate models and datasets need evaluation that remains independent of how they were produced. We present CheckMAITE, a composable computer-vision test-and-evaluation (T&E) framework that separates candidate development from evaluation. MIT Lincoln Laboratory’s Modular AI Trustworthy Engineering (MAITE) protocols provide common component interfaces through which CheckMAITE runs built-in and organization-defined tests. Human-readable reports, machine-readable records, and capability-defined feedback can be consumed by a machine client or reviewed by humans; they do not certify safety or authorize deployment. We evaluate CheckMAITE through deterministic integration demonstrations and illustrate an evaluation performed with CheckMAITE using a SeaDronesSee object detector. The results support a practical, reviewable assurance boundary for increasingly automated AI development.

Why Automated Development Needs a T&E Boundary

AI systems increasingly generate and revise candidate artifacts during model development, from research assistants to evaluator-guided program search (Lu et al. 2024; Schmidgall et al. 2025; Romera-Paredes et al. 2024; Jiang et al. 2025; Novikov et al. 2025). This motivates a stable evaluation boundary independent of the candidate-production method. A computer-vision assurance workflow may combine task performance, dataset analysis, robustness, explanation outputs, and review documentation rather than rely on one score (National Institute of Standards and Technology 2023; Arnold et al. 2019).

Here, a *candidate artifact* is a model or dataset submitted for evaluation. We use *assurance* to mean organization-selected evidence about that candidate, not certification, a safety determination, or deployment approval. CheckMAITE neither creates nor manages the candidate. Our research question is: *Can one computer-vision T&E harness run heterogeneous, organization-selected tests and return reviewable evidence through a common machine-client interface, independently of the candidate-production method?*

CheckMAITE is an open-source Python framework developed as an integration point for tools in the Joint AI

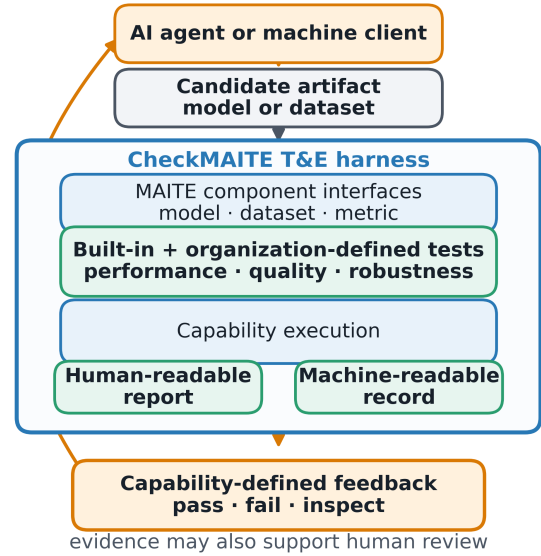


Figure 1: MAITE protocols provide common component interfaces. CheckMAITE executes built-in or organization-defined tests and returns capability-defined evidence. That evidence may inform a client or reviewer, but it does not authorize deployment.

Test Infrastructure Capability (JATIC) program (OpenTeams and CheckMAITE Contributors 2026). It accepts MAITE-conforming components and adds capability execution, reporting, analytics, and reuse layers. This paper contributes: (1) an assurance framing that separates candidate production, evaluation, and decision authority; (2) executable demonstrations of heterogeneous built-in and organization-defined tests returning evidence through one interface; and (3) a maritime object-detection profile illustrating how multiple criteria can produce bounded feedback.

A Composable Assurance Harness

Figure 1 shows the boundary from candidate components to assurance evidence. The harness combines component contracts, test capabilities, execution, human-readable reports,

and machine-readable records.

MAITE as the composability interface. MIT Lincoln Laboratory’s MAITE defines Python structural protocols for models, datasets, metrics, data loaders, and augmentations (MIT Lincoln Laboratory and MAITE Contributors 2026). Compatible components need not inherit from a common implementation. CheckMAITE capabilities consume these protocols, allowing the candidate implementation to remain separate from the test.

Composable assurance tests. CheckMAITE’s *Capability* abstraction packages the configuration, execution, and outputs of a computer-vision test (OpenTeams and CheckMAITE Contributors 2026). Built-in capabilities include model evaluation over MAITE-conforming inputs, dataset analyses from DataEval (DataEval Contributors 2026), and perturbation-robustness tests from Kitware’s Natural Robustness Toolkit (NRTK) (Hu et al. 2024). Organizations can add versioned capabilities through plug-ins. Interface compatibility permits execution; it does not establish that a contributed test is trustworthy or scientifically valid.

Evidence and identity. CheckMAITE records the capability, configuration, and caller-supplied component identifiers and metadata (OpenTeams and CheckMAITE Contributors 2026). Those identifiers bind evidence to artifact contents only if the surrounding integration maintains that relationship correctly. Tests can emit typed human-readable reports and flat analytics records for comparison and automation. Capability authors may additionally define machine-readable criteria or status fields; CheckMAITE transports but does not standardize their meaning.

Feedback without authorization. A machine client can request a permitted test and consume its evidence. Test-defined labels such as `pass`, `fail`, and `inspect` may inform revision, further evaluation, or escalation, but are not universal safety judgments. The surrounding system remains responsible for request authorization, artifact binding, access control, interpretation, and deployment decisions.

Evaluation of CheckMAITE

We evaluate CheckMAITE at two levels. First, deterministic demonstrations test whether the framework runs heterogeneous capabilities, returns human- and machine-readable evidence, and loads an organization-defined capability. These checks primarily use synthetic candidate artifacts. Second, a SeaDronesSee case study illustrates an assurance evaluation performed with CheckMAITE on a non-synthetic candidate model and development subset. The first level evaluates CheckMAITE; the second evaluates a candidate with CheckMAITE and is not a framework benchmark or deployment certification.

Framework integration demonstrations. Synthetic checks execute DataEval and NRTK capabilities over MAITE-conforming inputs and retain reports and analytics records. A plug-in demonstration loads and runs an organization-defined capability whose declared major

Check	Observed	Illustrative criterion	Result
Aggregate mAP@50	0.853	≥ 0.80	Met
Brightness retention	0.984	≥ 0.95	Met
Life-saving-appliance AP@50	0.550	≥ 0.60	Not met
Overall profile status	—	Organization-defined rule	<code>inspect</code>

Table 1: SeaDronesSee development-profile results. Criteria were not preregistered or validated for deployment.

API version is compatible, while recording an incompatible declaration as a loading problem. In a separate deterministic-client demonstration, an external allowlist rejects an unpermitted request; a permitted request invokes CheckMAITE and returns a run receipt. These results establish interface and evidence plumbing, not adaptive-agent behavior or candidate quality.

SeaDronesSee assurance profile. We applied CheckMAITE to an author-retained custom P2 detector configuration derived from YOLO11n (Ultralytics 2026) on a deterministic 78-image development subset of SeaDronesSee Object Detection v2 (Kiefer et al. 2023) containing 382 annotations across all five target classes. An *assurance profile* combines selected checks, criteria, and an overall status rule. This profile combined model evaluation, a three-point NRTK brightness sweep, and an organization-defined minimum class criterion.

Mean average precision at an intersection-over-union threshold of 0.50 (mAP@50) was 0.853, meeting the illustrative 0.80 criterion. Brightness retention, defined as the minimum sweep mAP@50 divided by the factor-1.0 sweep value across multipliers 0.75, 1.0, and 1.25, was 0.984 and met the 0.95 criterion. Average precision at the same threshold (AP@50) for the life-saving-appliance class was 0.550, below the 0.60 criterion. Under the organization-defined rule, that shortfall produced overall status `inspect`.

The profile illustrates how a class-specific criterion can expose a concern hidden by a passing aggregate criterion. Its result contains a suggested investigation of the class result and development-set coverage; no adaptive agent consumed the profile in this demonstration. Maritime imagery, illumination variation, uneven class coverage, and class-specific behavior make this a useful challenged-context example. CheckMAITE operates here at a development/validation gate; humans and organizations still select tests, interpret `inspect`, and authorize subsequent action.

Evidence Boundaries and Limitations

The demonstrations establish integration, not the validity or completeness of any assurance method. The client is deterministic and non-adaptive; we do not evaluate hostile agents, operational monitoring or deployment, sociotechnical outcomes, or a complete safety case. Plug-ins execute as trusted code, and retained evidence is neither immutable nor tamper-evident. Reuse depends on caller-supplied identities

remaining bound to unchanged content.

The SeaDronesSee subset and thresholds are illustrative. Retention is normalized to the NRTK factor-1.0 value (0.858), not the separate standard-evaluation value (0.853); the retained evidence does not include predictions needed to explain their 0.005193 difference. The profile used CheckMAITE 0.2.2 and NRTK 1.0.3, whereas the synthetic demonstrations use a post-0.3 CheckMAITE snapshot and NRTK 1.0.4. The checkpoint, images, labels, and selected sample identifiers are not distributed. The profile JSON includes three CheckMAITE Run identifiers, but they do not resolve to retained Run records or appear in the frozen analytics store. The retained aggregates, hashes, and report support arithmetic and file-integrity review, not independent verification of profile execution.

Assurance Context and Related Work

Agent benchmarks and program-search systems demonstrate dependence on executable evaluator feedback (Huang et al. 2024; Chan et al. 2025; Romera-Paredes et al. 2024; Jiang et al. 2025; Novikov et al. 2025). CheckMAITE does not generate candidates or claim novelty for the individual analyses. Its contribution is the common computer-vision execution-and-evidence boundary through which selected analyses can be invoked and reviewed.

In 2023, the Chief Digital and Artificial Intelligence Office described its establishment of JATIC as its “keystone T&E effort”: interoperable software capabilities for AI algorithm evaluation, robustness, adversarial resilience, human understanding of outputs, competence, and integration with different Department of Defense AI/ML pipelines (Martell 2023). Developed under JATIC, MAITE supplies the common component protocols; DataEval and NRTK supply particular analyses; CheckMAITE connects such capabilities through one T&E workflow. The agent-facing machine-client use in this paper is an application of that workflow, not a function attributed to JATIC or MAITE.

Conclusion

Increasing automation motivates separating candidate production from a stable evaluation interface. CheckMAITE uses MAITE protocols to compose built-in and organization-defined computer-vision tests and expose human-readable reports, machine-readable records, and capability-defined feedback through a common workflow. The demonstrations establish integration and evidence transport, not adaptive behavior or safety. In the SeaDronesSee case study, aggregate performance and brightness retention met illustrative criteria, while a class-level shortfall yielded *inspect*. Such evidence can inform revision or escalation, but does not certify safety or authorize deployment. CheckMAITE’s contribution is therefore a shared, reviewable assurance boundary for increasingly automated computer-vision development.

References

- Arnold, M.; Bellamy, R. K. E.; Hind, M.; Houde, S.; Mehta, S.; Mojsilović, A.; Nair, R.; Ramamurthy, K. N.; Olteanu, A.; Piorkowski, D.; Reimer, D.; Richards, J.; Tsay, J.; and Varshney, K. R. 2019. FactSheets: Increasing Trust in AI Services through Supplier’s Declarations of Conformity. *IBM Journal of Research and Development*, 63(4/5): 6:1–6:13.
- Chan, J. S.; Chowdhury, N.; Jaffe, O.; Aung, J.; Sherburn, D.; Mays, E.; Starace, G.; Liu, K.; Maksin, L.; Patwardhan, T.; Weng, L.; and Madry, A. 2025. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *The Thirteenth International Conference on Learning Representations*.
- DataEval Contributors. 2026. DataEval [software]. <https://github.com/aria-ml/dataeval>. Version 1.0.6 used by the synthetic artifact; accessed 2026-08-28.
- Hu, B.; RichardWebster, B.; Tunison, P.; Veenhuis, E.; Ravichandran, B.; Lynch, A.; Crowell, S.; Genova, A.; Bolea, V.; Jourdain, S.; and Whitesell, A. 2024. NRTK: An Open Source Natural Robustness Toolkit for the Evaluation of Computer Vision Models. In *Assurance and Security for AI-Enabled Systems*. SPIE.
- Huang, Q.; Vora, J.; Liang, P.; and Leskovec, J. 2024. MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 20271–20309. PMLR.
- Jiang, Z.; Schmidt, D.; Srikanth, D.; Xu, D.; Kaplan, I.; Jacenko, D.; and Wu, Y. 2025. AIDE: AI-Driven Exploration in the Space of Code. *arXiv preprint arXiv:2502.13138*.
- Kiefer, B.; Kristan, M.; Perš, J.; Žust, L.; Poiesi, F.; Andrade, F.; Bernardino, A.; et al. 2023. 1st Workshop on Maritime Computer Vision (MaCVi) 2023: Challenge Results. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops*, 265–302. Doi:10.1109/WACVW58289.2023.00033.
- Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.
- Martell, C. 2023. Chief Digital and Artificial Intelligence Officer: Statement for the Record. House Armed Services Committee, Subcommittee on Cyber, Information Technology, and Innovation. March 9, 2023.
- MIT Lincoln Laboratory and MAITE Contributors. 2026. MAITE: Modular AI Trustworthy Engineering [software]. <https://github.com/mit-ll-ai-technology/maite>. Version 0.9.2 used by the artifact; accessed 2026-08-28.
- National Institute of Standards and Technology. 2023. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology.
- Novikov, A.; Vū, N.; Eisenberger, M.; et al. 2025. AlphaEvolue: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131*.
- OpenTeams and CheckMAITE Contributors. 2026. CheckMAITE [software repository]. <https://github.com/openteams-ai/checkmaite>. Synthetic artifact version 0.3.0.post19.dev0+4f94bd1, commit 4f94bd1c8e32927961299ca5a38767396e58c79f; accessed 2026-08-28.
- Romera-Paredes, B.; Barekatin, M.; Novikov, A.; et al. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625(7995): 468–475.
- Schmidgall, S.; Su, Y.; Wang, Z.; et al. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. *arXiv preprint arXiv:2501.04227*.
- Ultralytics. 2026. Ultralytics YOLO11. <https://docs.ultralytics.com/models/yolo11/>. Ultralytics version 8.4.90 used by the retained profile; accessed 2026-08-28.